Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Spring, Year: 2024), B.Sc. in CSE (Day)

LAB REPORT NO: 01
Course Title: Artificial Intelligence Lab
Course Code: CSE-316 Section: 211 D2

Lab Experiment Name: Implement BFS and DFS demonstration and Code

Student Details

	Name	ID
1.	Md Mahdi Hasan	211002069

Lab Date : 12-03-24
Submission Date : 19-03-24
Course Teacher's Name : Md. Naimul Pathan

[For Teachers use only: Don't Write Anything inside this box]

Lab Report Status

Marks: …… **Signature:.....**
Comments:..... **Date:.....**

1. TITLE OF THE LAB EXPERIMENT:

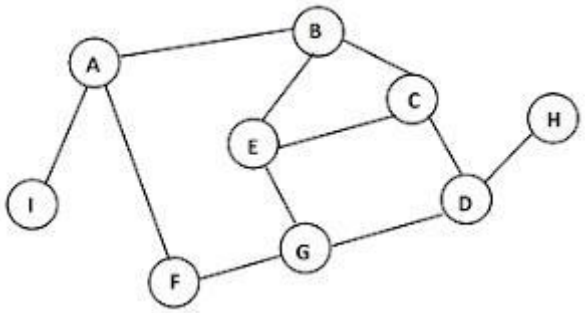
Implement BFS and DFS demonstration and Code.

2. OBJECTIVES:

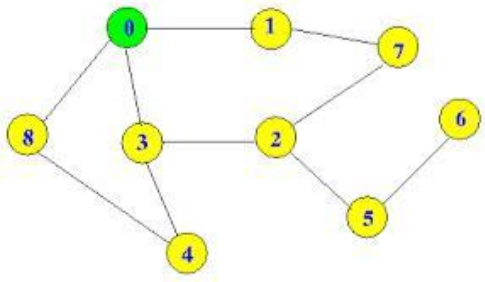
- The BFS and DFS algorithms for traversing a graph.
- BFS and DFS are fundamental algorithms in graph theory and are used for various tasks such as finding connected components, detecting cycles, and finding paths between vertices.

3. PROCEDURE:

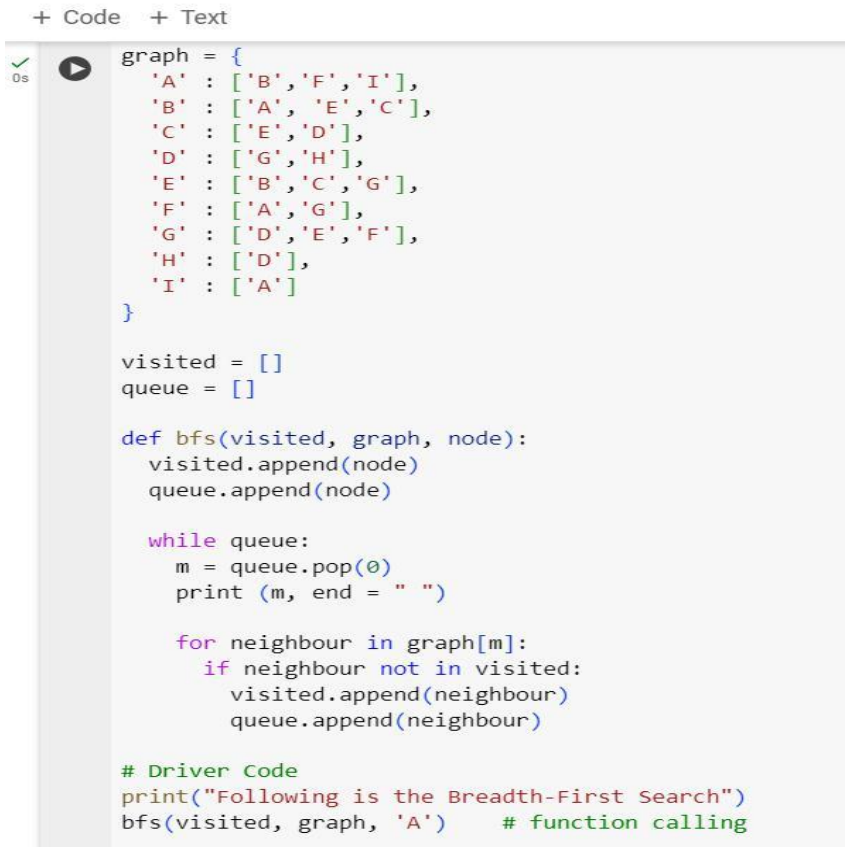
BFS: BFS specifically can be used to find the shortest path between two nodes in an unweighted graph. Breadth-First Search can be used as a traversal method to find all the neighboring nodes in a Peer-to-Peer Network. For example, BitTorrent uses Breadth-First Search for peer-to-peer communication.so that was all about the working of the Breadth - First Search algorithm.



DFS: DFS is often used as a subroutine in more complex algorithms. The outcome of a DFS traversal of a graph is a spanning. A spanning tree is a graph that is devoid of loops. To implement DFS traversal, you need to utilize a stack data structure with a maximum size equal to the total number of vertices in the graph.

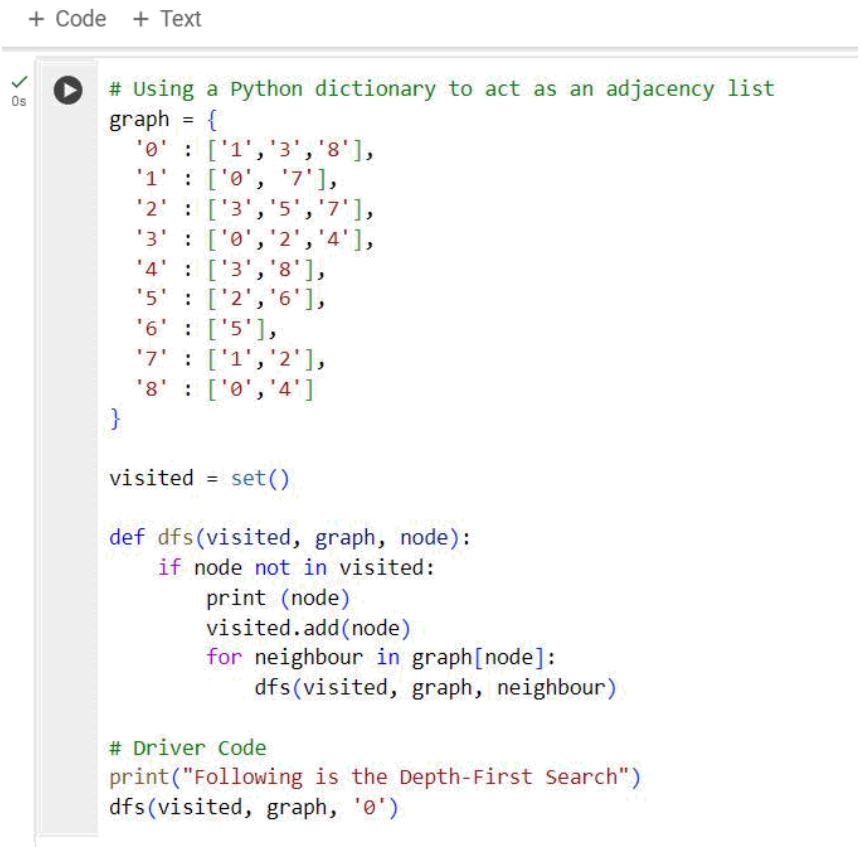


4. IMPLEMENTATION: BFS (Source Code):



```
graph = {
  'A' : ['B','F','I'],
  'B' : ['A', 'E','C'],
  'C' : ['E','D'],
  'D' : ['G','H'],
  'E' : ['B','C','G'],
  'F' : ['A','G'],
  'G' : ['D','E','F'],
  'H' : ['D'],
  'I' : ['A']
}

visited = []
queue = []

def bfs(visited, graph, node):
  visited.append(node)
  queue.append(node)

  while queue:
    m = queue.pop(0)
    print (m, end = " ")

    for neighbour in graph[m]:
      if neighbour not in visited:
        visited.append(neighbour)
        queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')    # function calling
```

DFS (Source Code):



```
# Using a Python dictionary to act as an adjacency list
graph = {
  '0' : ['1','3','8'],
  '1' : ['0', '7'],
  '2' : ['3','5','7'],
  '3' : ['0','2','4'],
  '4' : ['3','8'],
  '5' : ['2','6'],
  '6' : ['5'],
  '7' : ['1','2'],
  '8' : ['0','4']
}

visited = set()

def dfs(visited, graph, node):
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Depth-First Search")
dfs(visited, graph, '0')
```

5. **OUTPUT: BFS (output)**

6.

```
# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')    # function calling
```

```
Following is the Breadth-First Search
A B F I E C G D H
```

DFS (output):

```
Following is the Depth-First Search
0
1
7
2
3
4
8
5
6
```

6. DISCUSSION:

After completing this experiment, BFS explores all the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level. DFS explores as far as possible along each branch before backtracking. BFS guarantees to find the shortest path from the starting node to any other reachable node in an unweighted graph. DFS is not guaranteed to find the shortest path from the starting node to any other reachable node, but it's useful for tasks like topological sorting, finding strongly connected components, etc. each with its own advantages and use cases. Depending on the problem requirements, one might be preferred over the other.